



WP Activity Dashboard 1.0.1

DOCUMENTACIÓN TÉCNICA Y GUÍA DE IMPLEMENTACIÓN

JOEL VEGAS ROMERO

Índice

Introducción	3
¿Qué vas a lograr al finalizar este manual?	3
Arquitectura del Plugin.....	3
¿Por qué decimos que esto es "escalable y fácil de mantener"?	4
Hoja de Ruta: De cero a WP Activity Dashboard	4
Paso 1: El "Esqueleto" (El archivo principal).....	4
Paso 2: La "Memoria" (Base de Datos).....	4
Paso 3: El "Capturador" (Lógica de seguimiento).....	4
Paso 4: El "Cerebro" (Clase WAD_Data)	4
Paso 5: El "Escaparate" (UI y Gráficos)	4
Paso 6: El "Escaparate" (Interfaz de Usuario y Gráficos)	5
Paso 1: El "Esqueleto"	6
1.1. Objetivos en el paso 1.....	6
1.2. Preparación del Entorno	6
¿Qué herramientas necesitas?	6
La Estructura de Archivos	6
¿Por qué esta estructura es la correcta para exportar?	7
1.3. La Cabecera del Plugin: "El DNI"	7
Explicación	7
¿Por qué es crucial para la portabilidad?	8
1.4. Seguridad Fundamental: El "Portero" de tu paquete	8
¿Por qué es vital en un entorno exportable?	8
1.5. El Ensamblaje: Conectando las piezas	8
Explicación:	9
Paso 2: La "Memoria" (Base de Datos) y la Estructura.....	10
2.1. La Memoria: Creación de la Tabla	10
Explicación:	10
Explicación:	11
Paso 3: El "Capturador" (Lógica de seguimiento).....	11
3.1. Los filtros de seguridad (El Portero)	11
3.2. Preparando la identidad del artículo.....	12
Explicación:	12
3.3. Contador Total: Histórico de visitas.....	13
Explicación:	13
3.4. Contador Único Diario (Inteligencia con Cookies).....	13

Explicación:	13
3.5. Guardado inteligente en Base de Datos	14
Explicación:	14
Código función wad_track_post_views.....	14
3.6: Registro y Seguimiento de Visitas (wad_track_post_views)	15
Explicación:	15
Paso 4: El "Cerebro" (Clase WAD_Data)	17
Explicación:	17
4.1. Estructura de includes/class-wad-data.php	18
4.2. Desglose funcional de WAD_Data	18
4.2.1 Función get_content_distribution()	21
Explicación:	21
4.2.2 Función get_recent_posts_activity()	21
Explicación:	22
4.2.3. Función get_most_viewed_posts()	22
Explicación:	22
4.2.4. Función get_comment_status()	23
Explicación:	23
4.2.5. Función get_users_by_role()	23
Explicación:	23
4.2.6. Función get_total_views()	24
Explicación:	24
4.2.7. Función get_daily_unique_views().....	24
Explicación:	25
Paso 5: El "Escaparate" (Interfaz de Usuario y Gráficos)	26
5.1. Registrar la página de menú	26
Explicación:	27
5.2. Encolar scripts y preparar los datos	27
Explicación:	28
5.3. Renderizar el HTML	30
5.4. Cargar y ejecutar las instrucciones de los gráficos (JS).....	31
Explicación:	35
Paso 6: El "Escaparate" (Estilos CSS).....	36
6.1. ¿Dónde ponerlo?	36
6.2. El código CSS.....	36

Introducción

Bienvenido a la documentación técnica de **WP Activity Dashboard**. Este manual ha sido diseñado con un objetivo claro: desglosar, paso a paso, cada línea del código fuente para que puedas comprender la lógica detrás de cada funcionalidad.

El enfoque no es solo que copies y pegues, sino que **entiendas el "por qué"** de cada decisión.

Al finalizar esta guía, habrás adquirido una base sólida para el desarrollo de plugins profesionales, combinando la potencia de **PHP** en el servidor con la interactividad de **JavaScript** en el navegador.

¿Qué vas a lograr al finalizar este manual?

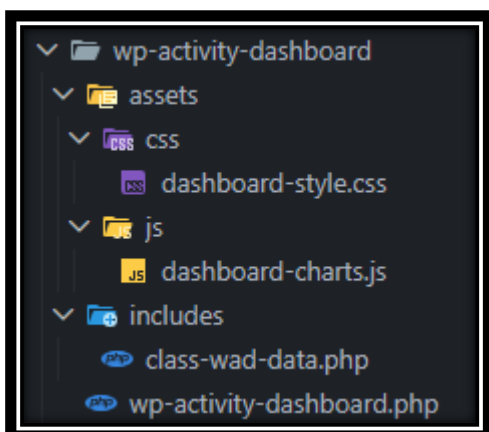
- **Entender la arquitectura** de un plugin de WordPress.
- **Dominar la persistencia de datos** usando wpdb y tablas personalizadas.
- **Integrar analítica de datos** con visualización interactiva mediante Chart.js.
- **Aprender buenas prácticas** de seguridad y escalabilidad en el desarrollo de software.

Arquitectura del Plugin

El punto de entrada (wp-activity-dashboard.php) es el "Recepcionista" de la cocina: Es el encargado de recibir todas las órdenes. Cuando alguien entra en tu página web, este archivo es el primero en saludar y decidir qué herramientas necesita usar para cumplir el pedido.

La lógica de negocio (includes/) es la "Despensa y los Chef": Aquí es donde está la receta secreta y los ingredientes procesados. No importa si los platos cambian, aquí es donde se prepara todo el trabajo pesado. Al tenerlo separado, si un día quieres cambiar la receta de una salsa, no tienes que remodelar toda la cocina, solo vas a este estante específico.

La presentación (assets/) es el "Emplatado": Es la parte final que ve el cliente (el diseño, los colores, los gráficos). Es el encargado de que el plato final se vea bonito y atractivo antes de que llegue a la mesa del usuario.



¿Por qué decimos que esto es "escalable y fácil de mantener"?

Porque si mañana quieres **cambiar el color de los gráficos** (la presentación el archivo "assets/), no corremos el riesgo de romper accidentalmente el código que calcula las visitas (la lógica del negocio el cual es el archivo "includes/").

Hoja de Ruta: De cero a WP Activity Dashboard

Paso 1: El "Esqueleto" (El archivo principal)

Qué hacer: Crear la carpeta del plugin y el archivo wp-activity-dashboard.php.

Objetivo: Lograr que WordPress reconozca el plugin en el panel de control. Añadir la cabecera (nombre, versión, autor) y el bloque de seguridad (ABSPATH).

Por qué: Es la prueba de fuego; sin esto, no hay nada que gestionar.

Paso 2: La "Memoria" (Base de Datos)

Qué hacer: Programar la función de activación (wad_activate).

Objetivo: Crear la tabla wad_daily_views en la base de datos de WordPress.

Por qué: Los datos son el alma de este plugin. Si no tenemos una tabla donde guardarlos, no tenemos nada que mostrar en los gráficos.

Paso 3: El "Capturador" (Lógica de seguimiento)

Qué hacer: Implementar wad_track_post_views.

Objetivo: Detectar cuando alguien visita una página y registrarlo (usando cookies para evitar registros duplicados).

Por qué: Ahora que tienes la tabla, necesitas llenarla de información real cada vez que un usuario navega por el sitio.

Paso 4: El "Cerebro" (Clase WAD_Data)

Qué hacer: Crear includes/class-wad-data.php.

Objetivo: Escribir las funciones que consultan a WordPress y devuelven los datos limpios (cuántos posts, cuántos usuarios, qué páginas son las más visitadas).

Por qué: Es el paso que separa un script(archivo) desordenado de un **programa profesional**. Centralizamos el trabajo aquí para que sea fácil de mantener.

Paso 5: El "Escaparate" (UI y Gráficos)

Qué hacer: Crear la página de menú (wad_register_admin_menu), cargar los estilos CSS, el archivo JS y renderizar el HTML con los canvas(graficos) de Chart.js.

Objetivo: Visualizar los datos que hemos estado recolectando en los pasos anteriores.

Por qué: Es la recompensa. Aquí es donde los datos se convierten en información útil para el dueño del sitio.

Paso 6: El "Escaparate" (Interfaz de Usuario y Gráficos)

Qué hacer: Crear la página de menú (wad_register_admin_menu), cargar los estilos CSS, el archivo JS y renderizar el HTML con los elementos canvas necesarios para que Chart.js dibuje los gráficos.

Objetivo: Visualizar de forma clara y profesional todos los datos que hemos estado recolectando en los pasos anteriores.

Por qué: Es la recompensa del trabajo. Aquí es donde los números fríos almacenados en la base de datos se convierten en información útil y accionable para el dueño del sitio web.

Paso 1: El "Esqueleto"

1.1. Objetivos en el paso 1

Qué vamos a crear: La estructura de carpetas y el archivo principal.

Objetivo: Conseguir que WordPress reconozca nuestro plugin portable.

Por qué: Es la base sobre la que se construye toda la lógica; sin esto, WordPress no puede ni cargar ni gestionar ninguna de nuestras funciones.

1.2. Preparación del Entorno

Para crear tu propio plugin, no necesitas ser un experto, pero sí necesitas tener las herramientas adecuadas para ello.

Sigue estos pasos para dejar todo listo.

¿Qué herramientas necesitas?

Un editor de código (Tu escritorio): Te sugerimos [Visual Studio Code \(VS Code\)](#). Es una herramienta gratuita, muy potente y utilizada por programadores de todo el mundo. Te ayuda a escribir código con colores, lo que facilita mucho detectar errores.

Un entorno de WordPress (Tu laboratorio): Necesitas un lugar donde instalar y probar el plugin. Puedes instalar WordPress directamente en tu ordenador usando herramientas como [LocalWP](#). Esto crea un sitio web privado en tu máquina donde puedes hacer todas las pruebas que quieras sin miedo a romper nada.

La Estructura de Archivos

Cuando trabajes en tu ordenador, todo tu proyecto debe vivir dentro de una carpeta raíz única. Esta carpeta es la que finalmente convertirás en un archivo .zip para compartir entre otros desarrolladores o clientes.

La estructura debe ser exactamente esta:

- **wp-activity-dashboard** (Carpeta raíz del plugin)
 - **wp-activity-dashboard.php** (*Archivo principal*)
 - **includes/** (*Lógica de negocio (programación) y procesamiento de datos*)
 - **assets/** (*Recursos visuales*)
 - **css/**
 - dashboard-style.css
 - **js**
 - dashboard-charts.js

¿Por qué esta estructura es la correcta para exportar?

Independencia: Al agrupar todo en la carpeta wp-activity-dashboard, garantizamos que WordPress no entrará en conflicto con otros plugins al importar el archivo .zip.

Modularidad: Separar los archivos en includes/ y assets/ permite que tu código sea escalable. Si en el futuro añades más funcionalidades, solo tendrás que añadir archivos en estas subcarpetas sin ensuciar el archivo principal.

Facilidad de Instalación: Al comprimir **solo** la carpeta wp-activity-dashboard en formato .zip, el usuario podrá ir a *Plugins > Añadir nuevo > Subir plugin*, seleccionar tu archivo, y WordPress lo instalará perfectamente, creando él mismo la ruta necesaria en su servidor.

Consejo de Desarrollador: Nunca entregues archivos sueltos. Asegúrate siempre de que tu carpeta raíz contenga todos los subdirectorios necesarios. Cuando comprimas la carpeta para exportarla, haz clic derecho sobre la carpeta wp-activity-dashboard y selecciona "Comprimir" o "Enviar a carpeta comprimida en zip". **Ese** es tu producto final.

1.3. La Cabecera del Plugin: "El DNI"

¿Dónde? Abre tu archivo **wp-activity-dashboard.php** con VS Code.

Cuando entregas un plugin en formato .zip, este archivo es el primer punto de contacto. WordPress no "lee" el código lógico (no lee el código php) al principio, sino que **escanea** esta cabecera para identificar el paquete.

¿Qué hacer? Pega el siguiente bloque de código al principio del archivo:

```
<?php
/**
 * Plugin Name: WP Activity Dashboard
 * Plugin URI: https://tusitio.com/
 * Description: Un panel de control elegante para visualizar las estadísticas de tu sitio WordPress con gráficos interactivos
 * Version: 2.0.0
 * Author: Ingeniero de Datos (Tú)
 * Author URI: https://tusitio.com/
 * Text Domain: wp-activity-dashboard
 */
```

Explicación

Aunque parezcan simples comentarios, para WordPress son "**Metadatos del Plugin**". Piensa en ellos como la **etiqueta de información** de un producto en un supermercado.

Plugin Name: Es el nombre que aparecerá en negrita en tu lista de plugins.

Version: Es fundamental para llevar el control. Si mañana decides mejorar tu plugin, cambiarás este número a 1.0.1, y así WordPress podrá saber si tienes la última versión instalada o si necesitas una actualización.

Author: Es tu sello de identidad; le dice al mundo quién ha construido este plugin.

¿Por qué es crucial para la portabilidad?

Registro automático: Al subir el **.zip**, WordPress extrae esta información y registra el plugin en su base de datos. Sin estos metadatos, el sistema no sabría qué nombre darle o cómo mostrarlo en el panel de administrador.

Gestión de Versiones: Si realizas una actualización en un futuro, WordPress comparará la Versión de tu **.zip** con la instalada para notificar al usuario que debe actualizar. Es la clave para mantener un ciclo de vida profesional del software.

Identificación del Autor: Es tu firma técnica. Permite que, aunque el plugin pase de mano en mano (o de servidor en servidor), siempre se sepa quién es el desarrollador original.

1.4. Seguridad Fundamental: El "Portero" de tu paquete

Como tu plugin será **exportado e instalado en servidores que no controlas**, es obligatorio blindar el archivo desde la primera línea.

¿Dónde? Abre tu archivo **wp-activity-dashboard.php** con VS Code.

¿Qué hacer? Añade esto justo después de la cabecera:

```
// Evitar acceso directo
if ( ! defined( 'ABSPATH' ) ) {
    exit;
}
```

¿Por qué es vital en un entorno exportable?

Al distribuir un archivo, estás "abriendo la puerta" de tu código a miles de configuraciones de servidor.

La constante ABSPATH: Esta constante es definida por el núcleo de WordPress cuando el plugin es cargado correctamente dentro del flujo del sistema.

Bloqueo de acceso directo: Si un usuario malintencionado intenta acceder a tu archivo mediante una URL directa (ej: `tusitio.com/wp-content/plugins/wp-activity-dashboard/wp-activity-dashboard.php`), nuestro código detectará que ABSPATH no existe y ejecutará un `exit` inmediato.

Nota: Incluir este bloque de seguridad es una norma estándar en los repositorios oficiales de WordPress. Al hacerlo, garantizas que tu **plugin sea seguro por diseño**, sin importar en qué servidor termine instalado.

1.5. El Ensamblaje: Conectando las piezas

Hasta ahora, hemos registrado el plugin y lo hemos protegido. Pero recuerda nuestra estructura: tenemos un "cerebro" en `includes/` y unos "recursos" en `assets/`. El archivo principal `wp-activity-dashboard.php` debe ser el puente que los conecte.

¿Qué hacer? Justo debajo del bloque de seguridad (exit), añade este código para dar "GPS" y "memoria" al plugin:

```
// Definir constantes del plugin
define( 'WAD_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );
define( 'WAD_PLUGIN_URL', plugin_dir_url( __FILE__ ) );
// Nombre de la tabla personalizada para almacenar visitas diarias
define( 'WAD_TABLE_DAILY_VIEWS', $wpdb->prefix . 'wad_daily_views' );
```

Explicación:

Estas líneas, son los cimientos del plugin:

- **define('WAD_PLUGIN_DIR', ...):**
 - **¿Qué hace?:** Guarda la ubicación exacta de tu carpeta en una "variable constante" (un valor que no cambia).
 - **Por qué importa:** Como el plugin es **portable**, no sabemos dónde lo instalará el usuario. Esta función le pregunta al servidor: "*¿Dónde estoy instalado?*". Esto permite que el plugin siempre encuentre sus archivos, sin importar el servidor.
- **define('WAD_PLUGIN_URL', ...):** Igual que la anterior, pero para la web. Es la dirección que usaremos para decirle a WordPress dónde están tus iconos, estilos y scripts.
- **define('WAD_TABLE_DAILY_VIEWS', \$wpdb->prefix . 'wad_daily_views'):** Aquí estamos nombrando nuestra tabla. Usamos \$wpdb->prefix (que suele ser wp_) para que, si el usuario tiene otros plugins, nuestra tabla no choque con ellos. Es una convención de limpieza y profesionalidad.

Paso 2: La "Memoria" (Base de Datos) y la Estructura

En este paso, nuestro objetivo es crear un lugar seguro donde el plugin pueda "anotar" las visitas y conectar su parte inteligente (la lógica) con el archivo principal.

2.1. La Memoria: Creación de la Tabla

Ahora que el plugin sabe dónde se aloja, vamos a crear el "almacén" donde se guardarán los datos para luego generar los gráficos.

¿Qué hacer? Añade este código a continuación:

```
// Crear tabla al activar el plugin
function wad_activate() {
    global $wpdb;
    $table_name = WAD_TABLE_DAILY_VIEWS;
    $charset_collate = $wpdb->get_charset_collate();
    $sql = "CREATE TABLE IF NOT EXISTS $table_name (\n
        id BIGINT(20) UNSIGNED NOT NULL AUTO_INCREMENT,\n
        post_id BIGINT(20) UNSIGNED NOT NULL,\n
        view_date DATE NOT NULL,\n
        views BIGINT(20) UNSIGNED NOT NULL DEFAULT 1,\n
        PRIMARY KEY (id),\n
        UNIQUE KEY post_date (post_id, view_date)\n
    ) $charset_collate;";
    require_once ABSPATH . 'wp-admin/includes/upgrade.php';
    dbDelta( $sql );
}
register_activation_hook( __FILE__, 'wad_activate' );
```

Explicación:

function wad_activate(): Es la instrucción que nos permite crear la base de datos.

\$charset_collate: Esto asegura que si tu web está en español, la tabla entienda correctamente tildes y caracteres especiales (como la ñ).

CREATE TABLE IF NOT EXISTS: Ordena: *"Crea esta tabla, pero solo si no existe ya"*. Esto evita borrar datos antiguos al actualizar.

dbDelta(\$sql): Esta es una herramienta inteligente de WordPress. Si en el futuro decides cambiar algo de tu tabla, dbDelta se encarga de modificarla sin que pierdas ni un solo dato que ya hayas guardado.

register_activation_hook: Es el "botón de inicio". Le dice a WordPress: *"Cuando el usuario presione 'Activar' en mi plugin, ejecuta la función wad_activate"*.

2.2. Incluyendo el Cerebro

Para mantener el código limpio y profesional, no escribiremos toda la lógica en un solo archivo. Hemos separado el "cerebro" (la clase WAD_Data) en la carpeta includes/.

¿Qué hacer? En el punto anterior hemos añadido esta línea:

```
require_once ABSPATH . 'wp-admin/includes/upgrade.php';
```

Explicación:

require_once: Piensa en esto como un "vínculo". Le decimos a nuestro archivo principal: "Ve a la carpeta includes y carga el archivo class-wad-data.php".

¿Por qué **_once**? La palabra **_once** es una capa de seguridad. Si por algún error el archivo intenta cargarse dos veces, el sistema se detiene antes de causar un fallo. Esto hace que tu plugin sea mucho más **estable**, evitando conflictos en servidores con configuraciones complejas.

Paso 3: El "Capturador" (Lógica de seguimiento)

Ahora que el plugin tiene "memoria" (la base de datos), necesitamos que empiece a trabajar. El "Capturador" es una función que vigila cada vez que un cliente lee un artículo, esa actividad se anota en nuestro sistema.

3.1. Los filtros de seguridad (El Portero)

Antes de empezar a contar, debemos asegurarnos de que la visita sea correcta. Si el administrador entra a editar una página, no queremos contar eso: son "Datos Sucios". Un mal dato puede desequilibrar los gráficos.

¿Qué hacer? Abre tu archivo wp-activity-dashboard.php y añade este bloque inicial:

```
// Función para registrar visitas únicas basadas en cookies
function wad_track_post_views() {
    if ( is_admin() || ! is_singular() || is_preview() ) {
        return;
    }
}
```

¿Qué hace?: Actúa como un portero de discoteca.

is_admin(): Si eres tú (el administrador) entrando al panel de control, no queremos contar eso como una visita real al blog.

! is_singular(): Si el usuario está viendo una página de "Categorías" o "Archivos", no queremos contarla aquí, **solo queremos contar entradas o páginas individuales**.

is_preview(): Si estás en modo borrador viendo cómo quedará tu artículo, no es una visita real.

return: Es la orden de "detente y no hagas nada más".

3.2. Preparando la identidad del artículo

Para que las estadísticas sean precisas, necesitamos saber exactamente qué estamos midiendo.

¿Qué hacer? Añade esto justo debajo del bloque anterior:

```
global $post, $wpdb;  
if ( ! $post ) {  
    return;  
}  
$post_id = $post->ID;  
$meta_key = '_wad_view_count';
```

Explicación:

global \$post, \$wpdb: Estamos "llamando" a dos herramientas maestras de WordPress. **\$post** contiene la información del artículo actual, y **\$wpdb** es nuestra herramienta para hablar directamente con la base de datos.

\$post_id: Es el número de identificación único de cada artículo en WordPress.

\$meta_key: Es el nombre de la variable (la etiqueta) donde guardaremos el contador. Usar un guion bajo _ al principio es una buena práctica en WordPress para ocultar variables técnicas de los usuarios comunes.

3.3. Contador Total: Histórico de visitas

Aquí guardamos el acumulado de toda la vida del artículo.

¿Qué hacer? Añade este bloque a continuación:

```
// ——— Contador total (páginas vistas) ———
$views = get_post_meta( $post_id, $meta_key, true );
if ( $views === '' ) {
    $views = 0;
}
$views++;
update_post_meta( $post_id, $meta_key, $views );
```

Explicación:

get_post_meta: Le preguntamos a la base de datos: "¿Qué valor tiene la etiqueta `_wad_view_count` en este post?".

if (\$views === ''): Si el artículo es nuevo, la base de datos no nos devolverá nada (un campo vacío). Con esta línea, convertimos ese vacío en un 0.

\$views++: Esta es la forma corta de decir `$views = $views + 1`. Sumamos una visita más.

update_post_meta: Guardamos el nuevo número, reemplazando al antiguo en la base de datos.

3.4. Contador Único Diario (Inteligencia con Cookies)

Evitamos que la misma persona infle los números; esto protege al servidor de colapsos por usuarios que recargan la página cientos de veces (datos falsos).

¿Qué hacer? Añade este bloque a continuación:

```
$cookie_name = 'wad_viewed_' . $post_id;
if ( ! isset( $_COOKIE[ $cookie_name ] ) ) {
    // Primera visita del usuario a este post en los últimos 24h
    setcookie( $cookie_name, '1', time() + DAY_IN_SECONDS, COOKIEPATH, COOKIE_DOMAIN );
}
```

Explicación:

\$cookie_name: Creamos un nombre de cookie único para este post específico (ej: `wad_viewed_123`).

! isset(\$_COOKIE[...]): El código pregunta: "¿Existe esta cookie en el navegador del usuario?". Si **NO** existe, entonces es la primera vez que visita este post hoy.

setcookie: Creamos el "sello" en el navegador del usuario. Le decimos: "Quédate aquí durante `DAY_IN_SECONDS` (24 horas)".

3.5. Guardado inteligente en Base de Datos

Finalmente, actualizamos nuestra tabla personalizada para los gráficos diarios.

¿Qué hacer? Completa la función con el cierre de llaves necesario:

```
// Guardar en tabla diaria
$today = current_time( 'Y-m-d' );
$table = WAD_TABLE_DAILY_VIEWS;
$wpdb->query( $wpdb->prepare(
    "INSERT INTO $table (post_id, view_date, views) VALUES (%d, %s, 1) ON DUPLICATE KEY UPDATE views = views + 1",
    $post_id,
    $today
) );
```

Explicación:

INSERT ... ON DUPLICATE KEY UPDATE: Esta es la joya de la corona.

INSERT: Intenta añadir una fila nueva para este día y este post.

ON DUPLICATE KEY UPDATE: Si la base de datos ve que ya existe una fila (porque ya alguien visitó el post hoy), en lugar de dar error, ejecuta la actualización: `views = views + 1`.

\$wpdb->prepare: Es vital por seguridad. Limpia los datos antes de enviarlos a la base de datos para evitar ataques malintencionados.

Código función wad_track_post_views

```
// Función para registrar visitas únicas basadas en cookies
function wad_track_post_views() {
    if ( is_admin() || ! is_singular() || is_preview() ) {
        return;
    }
    global $post, $wpdb;
    if ( ! $post ) {
        return;
    }
    $post_id = $post->ID;
    $meta_key = '_wad_view_count';
    // — Contador total (páginas vistas) —
    $views = get_post_meta( $post_id, $meta_key, true );
    if ( $views == '' ) {
        $views = 0;
    }
    $views++;
    update_post_meta( $post_id, $meta_key, $views );

    // — Contador único (por visitante) —
    $cookie_name = 'wad_viewed_' . $post_id;
    if ( ! isset( $_COOKIE[ $cookie_name ] ) ) {
        // Primera visita del usuario a este post en los últimos 24h
        setcookie( $cookie_name, '1', time() + DAY_IN_SECONDS, COOKIEPATH, COOKIE_DOMAIN );
        // Guardar en tabla diaria
        $today = current_time( 'Y-m-d' );
        $table = WAD_TABLE_DAILY_VIEWS;
        $wpdb->query( $wpdb->prepare(
            "INSERT INTO $table (post_id, view_date, views) VALUES (%d, %s, 1) ON DUPLICATE KEY UPDATE views = views + 1",
            $post_id,
            $today
        ) );
    }
}
add_action( 'wp', 'wad_track_post_views' );
```

3.6: Registro y Seguimiento de Visitas (wad_track_post_views)

Esta función, wad_track_post_views, es el **"Contador de Visitas"**. Su misión es observar cada vez que un usuario abre un artículo y sumar un punto a ese artículo en la base de datos de WordPress.

¿Qué hacer?

Pégala en tu archivo wp-activity-dashboard.php, justo después de la configuración de constantes (el "GPS" del plugin) y antes de la lógica de administración.

```
/**
 * Registrar visitas en las páginas y entradas individuales
 */
function wad_track_post_views() {
    // Si no es una entrada o página individual, es el panel de admin, o es una vista previa, salir
    if ( is_admin() || ! is_singular() || is_preview() ) {
        return;
    }

    global $post;
    if ( ! $post ) {
        return;
    }

    $post_id = $post->ID;
    $meta_key = '_wad_view_count';

    // Obtener visitas actuales
    $views = get_post_meta( $post_id, $meta_key, true );
    if ( $views === '' ) {
        $views = 0;
        delete_post_meta( $post_id, $meta_key );
        add_post_meta( $post_id, $meta_key, '1' );
    } else {
        $views++;
        update_post_meta( $post_id, $meta_key, $views );
    }
}
add_action( 'wp', 'wad_track_post_views' );
```

Explicación:

```
function wad_track_post_views() {
```

Qué hace: Define el inicio de la función. Es el nombre del "recolector" que pusimos a trabajar.


```
// Si no es una entrada o página individual, es el panel de admin, o es una vista previa, salir
if ( is_admin() || ! is_singular() || is_preview() ) {
    return;
}
```

Qué hace: Es el **Portero de Seguridad**.

is_admin(): Si es el administrador el que estas editando, no lo contamos.

! is_singular(): Si está en una página de categorías o archivos, no lo contamos.

is_preview(): Si se está viendo una vista previa de un post antes de publicar, no es una visita real.

return: Es la orden de "detente", no hagas nada si estamos en alguno de esos casos.

```
global $post;
if ( ! $post ) {
    return;
}
```

Qué hace: global \$post nos da acceso a la información del artículo que el usuario está viendo en ese momento. Si no hay un post (por ejemplo, en un error 404), la función se detiene.

```
$post_id = $post->ID;
$meta_key = '_wad_view_count';
```

Qué hace: Identifica al post con su número único (ID) y prepara la "cajita" (meta_key) donde guardaremos el total de visitas de ese post específico.

```
// Obtener visitas actuales
$view = get_post_meta( $post_id, $meta_key, true );
```

Qué hace: Pregunta a la base de datos: "¿Cuántas visitas tiene este artículo hasta ahora?".

```
if ( $views == '' ) {
    $views = 0;
    delete_post_meta( $post_id, $meta_key );
    add_post_meta( $post_id, $meta_key, '1' );
}
```

Qué hace: Si es la primera vez que alguien entra, el valor estará vacío. Aquí lo "limpiamos" y ponemos el contador a 1.

```
} else {
    $views++;
    update_post_meta( $post_id, $meta_key, $views );
}
```

Qué hace: Si el post ya tenía visitas, tomamos el número actual, le sumamos 1 (\$views++) y guardamos el nuevo resultado en la base de datos.

```
add_action( 'wp', 'wad_track_post_views' );
```

Qué hace: Este es el "Gancho". Le dice a WordPress: "Cada vez que alguien abra una página web pública (wp), ejecuta mi función de contador".

Paso 4: El "Cerebro" (Clase WAD_Data)

Como hemos visto en nuestra arquitectura (como tenemos organizada los directorios), el archivo principal es solo el "mensajero". Ahora necesitamos el "cerebro" que procesa la información para el Dashboard.

¿Qué hacer? Asegúrate de tener la línea que conecta ambos mundos en tu archivo principal:

```
// Incluir la clase de datos
require_once WAD_PLUGIN_DIR . 'includes/class-wad-data.php';
```

Explicación:

Al usar `require_once`, le decimos a WordPress que el archivo `class-wad-data.php` es una pieza fundamental del plugin. Si este archivo no se carga, el Dashboard no tendrá datos que mostrar y veremos errores.

4.1. Estructura de includes/class-wad-data.php

Este archivo contiene la clase WAD_Data. Imagina que es un **empleado experto en archivos** de tu sitio. Él no decide *qué* mostrar, pero cuando el Dashboard le pide un dato, él sabe exactamente dónde buscarlo, cómo sumarlo y cómo devolvértelo organizado.

4.2. Desglose funcional de WAD_Data

```
<?php
// Evitar acceso directo
if ( ! defined( 'ABSPATH' ) ) {
    exit;
}

/**
 * Clase encargada de recolectar datos estadísticos del sitio WordPress.
 */
class WAD_Data {

    public function get_content_distribution() {
        $posts_count = wp_count_posts( 'post' );
        $pages_count = wp_count_posts( 'page' );

        return array(
            'posts' => isset($posts_count->publish) ? (int) $posts_count->publish : 0,
            'pages' => isset($pages_count->publish) ? (int) $pages_count->publish : 0,
        );
    }

    public function get_comment_status() {
        $comments_count = wp_count_comments();

        return array(
            'approved' => isset($comments_count->approved) ? (int) $comments_count->approved : 0,
            'pending' => isset($comments_count->moderated) ? (int) $comments_count->moderated : 0,
            'spam' => isset($comments_count->spam) ? (int) $comments_count->spam : 0,
        );
    }

    public function get_users_by_role() {
        $user_counts = count_users();
        $roles = array();

        if ( isset( $user_counts['avail_roles'] ) ) {
            foreach ( $user_counts['avail_roles'] as $role => $count ) {
                $role_name = ucfirst( $role );
                if ( $role === 'administrator' ) $role_name = 'Administrador';
                if ( $role === 'editor' ) $role_name = 'Editor';
                if ( $role === 'author' ) $role_name = 'Autor';
                if ( $role === 'contributor' ) $role_name = 'Colaborador';
                if ( $role === 'subscriber' ) $role_name = 'Suscriptor';

                $roles[ $role_name ] = $count;
            }
        }

        return $roles;
    }
}
```



```

21  /**
22   * Obtiene las visitas únicas diarias para los últimos $days días.
23   * Devuelve un array asociativo donde la clave es la fecha (Y-m-d) y el valor es el número de visitas.
24   */
25  public function get_daily_unique_views( $days = 30 ) {
26      global $wpdb;
27      $table = WAD_TABLE_DAILY_VIEWS;
28      $date_from = date( 'Y-m-d', strtotime( "-{$days} days" ) );
29      $query = $wpdb->prepare(
30          "SELECT view_date, SUM(vIEWS) as views FROM $table WHERE view_date ≥ %s GROUP BY view_date ORDER BY view_date ASC",
31          $date_from
32      );
33      $results = $wpdb->get_results( $query, ARRAY_A );
34      // Rellenar todas las fechas del rango con 0 por defecto
35      $dates = array();
36      for ( $i = $days - 1; $i ≥ 0; $i-- ) {
37          $date_key = date( 'Y-m-d', strtotime( "-{$i} days" ) );
38          $dates[ $date_key ] = 0;
39      }
40      if ( $results ) {
41          foreach ( $results as $row ) {
42              if ( !isset( $dates[ $row['view_date'] ] ) ) {
43                  $dates[ $row['view_date'] ] = (int) $row['views'];
44              }
45          }
46      }
47      return $dates;
48  }
49
50  if ( ! function_exists( 'get_plugins' ) ) {
51      require_once ABSPATH . 'wp-admin/includes/plugin.php';
52  }
53
54  $all_plugins = get_plugins();
55  $active_plugins = get_option( 'active_plugins', array() );
56
57  $data = array(
58      'active_count' => 0,
59      'inactive_count' => 0,
60      'list' => array()
61  );
62
63  foreach ( $all_plugins as $plugin_path => $plugin_info ) {

```

```

162      $is_active = in_array( $plugin_path, $active_plugins );
163
164      if ( $is_active ) {
165          $data['active_count']++;
166      } else {
167          $data['inactive_count']++;
168      }
169
170      $data['list'][] = array(
171          'name' => $plugin_info['Name'],
172          'active' => $is_active
173      );
174  }
175
176  return $data;
177  }
178
179  }
180

```

4.2.1 Función `get_content_distribution()`

Ubicación: Dentro de la clase `WAD_Data`, en `includes/class-wad-data.php`.

```
public function get_content_distribution() {
    $posts_count = wp_count_posts( 'post' );
    $pages_count = wp_count_posts( 'page' );

    return array(
        'posts' => isset($posts_count->publish) ? (int) $posts_count->publish : 0,
        'pages' => isset($pages_count->publish) ? (int) $pages_count->publish : 0,
    );
}
```

Explicación:

`wp_count_posts('post')`: Es una función de WordPress que consulta cuántas entradas existen en tu base de datos. Guardamos el resultado en la variable `$posts_count`.

`return array(...)`: Preparamos un paquete de información (un "array" o lista) que contiene dos datos: posts y pages.

`isset($posts_count->publish)`: Esta es la variable de control. Pregunta: "¿Existe el conteo de posts publicados?".

`(int) ... : 0`: Si la respuesta es sí, lo convertimos a número entero (int). Si no existe nada, devolvemos 0 para que el sistema no dé error.

4.2.2 Función `get_recent_posts_activity()`

Ubicación: Dentro de la clase `WAD_Data` en `includes/class-wad-data.php`.

```
51 public function get_recent_posts_activity() {
52     global $wpdb;
53
54     $query = $wpdb->prepare(
55         "SELECT DATE(post_date) as date, COUNT(ID) as count
56         FROM {$wpdb->posts}
57         WHERE post_status = 'publish' AND post_type = 'post'
58         AND post_date ≥ %s
59         GROUP BY DATE(post_date)
60         ORDER BY date ASC",
61         date( 'Y-m-d', strtotime( '-30 days' ) )
62     );
63
64     $results = $wpdb->get_results( $query );
65
66     $dates = array();
67     for ( $i = 29; $i ≥ 0; $i-- ) {
68         $date_string = date( 'Y-m-d', strtotime( "-$i days" ) );
69         $dates[ $date_string ] = 0;
70     }
71
72     if ( $results ) {
73         foreach ( $results as $row ) {
74             if ( isset( $dates[ $row->date ] ) ) {
75                 $dates[ $row->date ] = (int) $row->count;
76             }
77         }
78     }
79
80     return $dates;
81 }
82
```

Explicación:

global \$wpdb; Llamamos a la herramienta que nos permite comunicarnos directamente con la base de datos de WordPress.

\$wpdb->prepare(...): Es una medida de seguridad. Preparamos una consulta SQL para pedirle a la base de datos: "Dame la fecha de creación y la cantidad de posts publicados en los últimos 30 días".

\$wpdb->get_results(\$query); Ejecutamos la consulta y almacenamos los resultados obtenidos en la variable \$results.

for (\$i = 29; \$i >= 0; \$i--): Creamos un bucle que genera una lista de los últimos 30 días con valor 0 por defecto. Esto garantiza que, aunque un día no se haya publicado nada, el gráfico muestre un 0 en lugar de un hueco vacío.

4.2.3. Función get_most_viewed_posts()

Ubicación: Dentro de la clase WAD_Data en includes/class-wad-data.php.

```
95 public function get_most_viewed_posts( $limit = 5 ) {
96     $args = array(
97         'post_type'      => array( 'post', 'page' ),
98         'post_status'    => 'publish',
99         'posts_per_page' => $limit,
100         'meta_key'       => '_wad_view_count',
101         'orderby'        => 'meta_value_num',
102         'order'          => 'DESC',
103         'fields'         => 'ids' // Solo obtener IDs por eficiencia
104     );
105
106     $query = new WP_Query( $args );
107     $most_viewed = array();
108
109     if ( $query->have_posts() ) {
110         foreach ( $query->posts as $post_id ) {
111             $most_viewed[] = array(
112                 'title' => get_the_title( $post_id ),
113                 'views' => (int) get_post_meta( $post_id, '_wad_view_count', true )
114             );
115         }
116     }
117
118     return $most_viewed;
119 }
120
```

Explicación:

\$args = array(...): Definimos los criterios de búsqueda. Le decimos a WordPress que solo queremos: posts y páginas, que estén publicados, ordenados por nuestro contador de visitas (_wad_view_count) de mayor a menor (DESC).

new WP_Query(\$args); Creamos una consulta personalizada usando los criterios anteriores para encontrar los artículos más populares.

'fields' => 'ids': Optimizamos el rendimiento pidiendo que nos devuelva solo los identificadores (IDs) de los posts, no toda la información, ahorrando recursos del servidor.

4.2.4. Función `get_comment_status()`

Ubicación: Dentro de la clase `WAD_Data` en `includes/class-wad-data.php`.

```
public function get_comment_status() {
    $comments_count = wp_count_comments();

    return array(
        'approved' => isset($comments_count->approved) ? (int) $comments_count->approved : 0,
        'pending'  => isset($comments_count->moderated) ? (int) $comments_count->moderated : 0,
        'spam'     => isset($comments_count->spam) ? (int) $comments_count->spam : 0,
    );
}
```

Explicación:

`wp_count_comments()`: Es una función interna de WordPress que ya tiene clasificados los comentarios (aprobados, pendientes, spam).

`return array(...)`: Crea un paquete con tres indicadores.

`isset($comments_count->approved) ? ... : 0`: Verifica si existe un conteo para ese estado. Si el resultado es positivo, lo devuelve como número entero; si no, devuelve un 0 para que el gráfico no tenga errores de lectura.

4.2.5. Función `get_users_by_role()`

Ubicación: Dentro de la clase `WAD_Data` en `includes/class-wad-data.php`.

```
32 public function get_users_by_role() {
33     $user_counts = count_users();
34     $roles = array();
35
36     if ( isset( $user_counts['avail_roles'] ) ) {
37         foreach ( $user_counts['avail_roles'] as $role => $count ) {
38             $role_name = ucfirst( $role );
39             if ( $role === 'administrator' ) $role_name = 'Administrador';
40             if ( $role === 'editor' ) $role_name = 'Editor';
41             if ( $role === 'author' ) $role_name = 'Autor';
42             if ( $role === 'contributor' ) $role_name = 'Colaborador';
43             if ( $role === 'subscriber' ) $role_name = 'Suscriptor';
44
45             $roles[ $role_name ] = $count;
46         }
47     }
48     return $roles;
49 }
```

Explicación:

`count_users()`: Devuelve un array con la cantidad de usuarios por cada rol existente.

`foreach`: Recorre cada rol detectado.

`ucfirst( $role )`: Pone la primera letra en mayúscula.

if (\$role === 'administrator') ...: Traduce las etiquetas técnicas de WordPress (que están en inglés) a términos en español más amigables para el usuario final del Dashboard.

4.2.6. Función get_total_views()

Ubicación: Dentro de la clase WAD_Data en includes/class-wad-data.php.

```
83  /**
84   * Obtiene el total de visitas de todo el sitio sumando los meta_keys
85   */
86   public function get_total_views() {
87       global $wpdb;
88       $total = $wpdb->get_var( "SELECT SUM(meta_value) FROM $wpdb->postmeta WHERE meta_key = '_wad_view_count'" );
89       return $total ? (int) $total : 0;
90   }
```

Explicación:

\$wpdb->get_var(...): Ejecuta una consulta SQL para extraer un único valor de la base de datos.

SELECT SUM(meta_value): Ordena a la base de datos sumando todos los números encontrados bajo la etiqueta _wad_view_count en la tabla de metadatos.

return \$total ? (int) \$total : 0: Devuelve la suma total. Si la base de datos no tiene datos todavía, devuelve 0.

4.2.7. Función get_daily_unique_views()

Ubicación: Dentro de la clase WAD_Data en includes/class-wad-data.php.

```
21  /**
22   * Obtiene las visitas únicas diarias para los últimos $days días.
23   * Devuelve un array asociativo donde la clave es la fecha (Y-m-d) y el valor es el número de visitas.
24   */
25   public function get_daily_unique_views( $days = 30 ) {
26       global $wpdb;
27       $table = WAD_TABLE_DAILY_VIEWS;
28       $date_from = date( 'Y-m-d', strtotime( "--{$days} days" ) );
29       $query = $wpdb->prepare(
30           "SELECT view_date, SUM(views) as views FROM $table WHERE view_date ≥ %s GROUP BY view_date ORDER BY view_date ASC",
31           $date_from
32       );
33       $results = $wpdb->get_results( $query, ARRAY_A );
34       // Rellenar todas las fechas del rango con 0 por defecto
35       $dates = array();
36       for ( $i = $days - 1; $i ≥ 0; $i-- ) {
37           $date_key = date( 'Y-m-d', strtotime( "--{$i} days" ) );
38           $dates[ $date_key ] = 0;
39       }
40       if ( $results ) {
41           foreach ( $results as $row ) {
42               if ( isset( $dates[ $row['view_date'] ] ) ) {
43                   $dates[ $row['view_date'] ] = (int) $row['views'];
44               }
45           }
46       }
47       return $dates;
48   }
49   if ( ! function_exists( 'get_plugins' ) ) {
50       require_once ABSPATH . 'wp-admin/includes/plugin.php';
51   }
52
53   $all_plugins = get_plugins();
54   $active_plugins = get_option( 'active_plugins', array() );
55
56   $data = array(
57       'active_count'  => 0,
58       'inactive_count' => 0,
59       'list'         => array()
60   );
```

```

2         foreach ( $all_plugins as $plugin_path => $plugin_info ) {
3             $is_active = in_array( $plugin_path, $active_plugins );
4
5             if ( $is_active ) {
6                 $data['active_count']++;
7             } else {
8                 $data['inactive_count']++;
9             }
10
11             $data['list'][] = array(
12                 'name' => $plugin_info['Name'],
13                 'active' => $is_active
14             );
15         }
16
17         return $data;
18     }

```

Explicación:

\$days = 30: Es un parámetro opcional. Si no se le dice nada, la función asume que queremos analizar el historial de los últimos 30 días.

\$table = WAD_TABLE_DAILY_VIEWS: Utiliza la constante definida en el archivo principal para saber exactamente a qué tabla de la base de datos debe consultar.

\$date_from = date('Y-m-d', ...): Calcula la fecha de hace 30 días atrás para usarla como punto de partida en nuestra búsqueda.

SELECT view_date, SUM(vIEWS) as views: Le ordena a la base de datos que sume todas las visitas de cada día específico.

\$wpdb->prepare(...): Es el escudo de seguridad. Convierte la variable de fecha en un formato seguro para evitar que alguien malintencionado pueda manipular la base de datos.

GROUP BY view_date: Esta instrucción es clave; le dice a la base de datos que junte todas las filas que pertenezcan al mismo día para poder sumarlas en un solo total diario.

ARRAY_A: Le indica a WordPress que nos devuelva los resultados como un array asociativo (donde podemos llamar a los datos por su nombre de columna, como view_date o views).

for (\$i = \$days - 1; \$i >= 0; \$i--): Es un bucle que crea un "calendario" virtual de los últimos 30 días y los inicializa todos en cero.

\$dates[\$date_key] = 0: Llena nuestro calendario virtual con ceros. Esto es vital para que, en el gráfico, los días sin actividad se vean como "0" y no como un error o una ausencia de datos.

if (isset(\$dates[\$row['view_date']])): Comprueba si el día que viene de la base de datos existe en nuestro calendario virtual.

\$dates[\$row['view_date']] = (int) \$row['views']: Si el día existe, reemplaza el "0" inicial por el número real de visitas que obtuvimos de la base de datos.

require_once ABSPATH . 'wp-admin/includes/plugin.php': Carga las funciones administrativas de WordPress necesarias para gestionar plugins, ya que no siempre están cargadas en el panel.

get_plugins(): Escanea toda la carpeta de plugins instalados y devuelve una lista detallada con sus nombres, versiones y autores.

get_option('active_plugins', array()): Consulta en la base de datos la lista de plugins que están marcados como "activos".

\$data = array(...): Crea la estructura de datos que vamos a devolver, inicializando contadores en cero.

foreach (\$all_plugins ...): Recorre cada uno de los plugins instalados uno por uno.

in_array(\$plugin_path, \$active_plugins): Verifica si el plugin que estamos analizando en este momento está dentro de la lista de activos.

\$data['active_count']++: Si la comprobación anterior fue positiva, suma uno al contador de plugins activos.

\$data['list'][]: Añade un elemento al paquete de datos final con el nombre del plugin y su estado (activo o inactivo) para que el Dashboard pueda mostrar la lista.

Paso 5: El "Escaparate" (Interfaz de Usuario y Gráficos)

Ahora configuraremos la parte visual. Este paso se divide en tres tareas: **registrar el menú**, **preparar los datos para el navegador** y **renderizar el HTML**.

5.1. Registrar la página de menú

Para que el usuario encuentre el panel en el lateral de WordPress:

```
/**
 * Registrar la página de menú en el admin de WordPress
 */
function wad_register_admin_menu() {
    add_menu_page(
        'Activity Dashboard',           // Título de la página
        'Panel de Actividad',           // Título del menú
        'manage_options',               // Capacidad requerida
        'wp-activity-dashboard',        // Slug del menú
        'wad_render_dashboard_page',    // Función que renderiza el HTML
        'dashicons-chart-area',        // Icono (Dashicon)
        30                              // Posición en el menú
    );
}
add_action( 'admin_menu', 'wad_register_admin_menu' );
```

Explicación:

add_menu_page(...): Es la función maestra de WordPress para añadir secciones al panel (barra lateral).

manage_options: Actúa como un "portero", bloqueando el acceso a cualquier usuario que no sea administrador.

add_action(...): Es la orden que le dice a WordPress: "Ejecuta mi función cuando estés creando el menú".

5.2. Encolar scripts y preparar los datos

Aquí es donde pasamos la información del servidor (PHP) al cliente (JavaScript) usando `wp_localize_script`.

```
/**
 * Encolar scripts y estilos solo en la página del plugin
 */
// Encolar scripts y estilos solo en la página del plugin
function wad_enqueue_admin_scripts( $hook ) {
    // Comprobar que estamos en la página correcta
    if ( $hook !== 'oplevel_page_wp-activity-dashboard' ) {
        return;
    }

    // Estilos del dashboard
    wp_enqueue_style( 'wad-dashboard-style', WAD_PLUGIN_URL . 'assets/css/dashboard-style.css', array(), '2.0.0' );

    // Chart.js desde CDN (versión 3.9.1 es muy estable)
    wp_enqueue_script( 'chartjs', 'https://cdn.jsdelivr.net/npm/chart.js@3.9.1/dist/chart.min.js', array(), null, true );

    // Nuestro script para inicializar los gráficos
    wp_enqueue_script( 'wad-dashboard-charts', WAD_PLUGIN_URL . 'assets/js/dashboard-charts.js', array( 'chartjs' ), '2.0.0', true );

    // Obtener los datos reales de WP usando nuestra clase
    $wad_data = new WAD_Data();
    $chart_data = array(
        'content_distribution' => $wad_data->get_content_distribution(),
        'comment_status'      => $wad_data->get_comment_status(),
        'users_by_role'       => $wad_data->get_users_by_role(),
        'recent_posts'        => $wad_data->get_recent_posts_activity(),
        'total_views'         => $wad_data->get_total_views(),
        'most_viewed'         => $wad_data->get_most_viewed_posts(),
        'plugins_data'        => $wad_data->get_plugins_data(),
        // Nuevos datos de visitas únicas diarias (últimos 30 días)
        'daily_unique_views'  => $wad_data->get_daily_unique_views(30)
    );

    // Pasar los datos de PHP a JavaScript
    wp_localize_script( 'wad-dashboard-charts', 'wadChartData', $chart_data );
}
add_action( 'admin_enqueue_scripts', 'wad_enqueue_admin_scripts' );
```

Esta función es el **"Preparador de suministros"**. Imagina que tu Dashboard es un restaurante: el código PHP es la cocina y el navegador del usuario es la mesa del cliente. Esta función prepara todos los platos (datos y archivos) para llevarlos a la mesa.

¿Qué hacer?

Debes pegar este bloque de código dentro de tu archivo principal `wp-activity-dashboard.php`. Es fundamental colocarlo después de haber definido las funciones básicas y antes de la parte visual, para que cuando la página cargue, los ingredientes (datos en nuestro caso) ya estén listos.

Explicación:

```
function wad_enqueue_admin_scripts( $hook ) {
```

Qué hace: Define el inicio de nuestra función. Es como ponerle un nombre al "equipo de trabajo" que preparará los suministros.

```
// Comprobar que estamos en la página correcta
if ( $hook !== 'oplevel_page_wp-activity-dashboard' ) {
    return;
}
```

Qué hace: Comprueba: "¿Estamos en la página de mi plugin?". Si la respuesta es NO, el código se detiene inmediatamente (return). Así evitamos que el plugin trabaje cuando no se le necesita.

```
// Estilos del dashboard
wp_enqueue_style( 'wad-dashboard-style', WAD_PLUGIN_URL . 'assets/css/dashboard-style.css', array(), '2.0.0' );
```

Qué hace: Le dice a WordPress: "Ve a buscar mi archivo de diseño .css y aplícalo". Es lo que hace que tu panel tenga colores, bordes y buena letra.

```
// Chart.js desde CDN (versión 3.9.1 es muy estable)
wp_enqueue_script( 'chartjs', 'https://cdn.jsdelivr.net/npm/chart.js@3.9.1/dist/chart.min.js', array(), null, true );
```

Qué hace: Descarga desde internet la librería Chart.js. Es como contratar a un artista profesional para que luego pueda dibujar los gráficos por nosotros.

```
// Nuestro script para inicializar los gráficos
wp_enqueue_script( 'wad-dashboard-charts', WAD_PLUGIN_URL . 'assets/js/dashboard-charts.js', array('chartjs'), '2.0.0', true );
```

Qué hace: Carga el archivo de instrucciones (dashboard-charts.js). El array('chartjs') le indica a WordPress: "No cargues este archivo hasta que el "artista" Chart.js esté presente".

```
// Obtener los datos reales de WP usando nuestra clase
$wad_data = new WAD_Data();
```

Qué hace: Llama al "Cerebro" (la clase WAD_Data). Es como darle el botón de encendido al analista para que empiece a buscar los datos en la base de datos.

```
$chart_data = array(
    'content_distribution' => $wad_data->get_content_distribution(),
    'comment_status'      => $wad_data->get_comment_status(),
    'users_by_role'       => $wad_data->get_users_by_role(),
    'recent_posts'        => $wad_data->get_recent_posts_activity(),
    'total_views'         => $wad_data->get_total_views(),
    'most_viewed'         => $wad_data->get_most_viewed_posts(),
    'plugins_data'        => $wad_data->get_plugins_data(),
    // Nuevos datos de visitas únicas diarias (últimos 30 días)
    'daily_unique_views' => $wad_data->get_daily_unique_views(30)
);
```

Qué hace: Crea una "caja" (un array) llamada \$chart_data y mete dentro todas las respuestas del analista (en nuestro caso recogerá todos los datos que están en la base de datos creada anteriormente, en otras palabras, estamos extrayendo todos los datos que hemos ido recogiendo). Cada línea es una pregunta a la base de datos: "¿Cuántos posts hay?", "¿Cuántas visitas?".

```
// Pasar los datos de PHP a JavaScript
wp_localize_script( 'wad-dashboard-charts', 'wadChartData', $chart_data );
```

Qué hace: Como el navegador (donde se ven los gráficos) no habla el idioma del servidor (PHP), esta función empaqueta todos nuestros datos en una variable llamada wadChartData que el lenguaje del navegador (JavaScript) sí entiende perfectamente.

```
}
add_action( 'admin_enqueue_scripts', 'wad_enqueue_admin_scripts' );
```

Qué hace: Es la orden definitiva. Le dice a WordPress: "Cada vez que vayas a cargar los archivos de administración, por favor, ejecuta esta función que acabo de crear".

5.3. Renderizar el HTML

Finalmente, creamos el diseño (canvas) donde se verán los gráficos.

```
/**
 * Renderizar el HTML de la página del panel
 */
function wad_render_dashboard_page() {
    if ( ! current_user_can( 'manage_options' ) ) {
        return;
    }
    ?>
    <div class="wrap wad-wrap">
        <h1><?php echo esc_html( get_admin_page_title() ); ?></h1>
        <p class="wad-description">Resumen visual de toda la actividad importante de tu sitio WordPress.</p>

        <div class="wad-grid">

            <!-- Tarjeta Nueva: Resumen de Visitas (KPI) -->
            <div class="wad-card wad-kpi-card">
                <h2>Visitas Totales</h2>
                <div class="wad-kpi-value" id="wadTotalViews">0</div>
                <p class="wad-kpi-desc">Páginas y Entradas vistas desde la instalación</p>
            </div>

            <!-- Tarjeta Nueva: Páginas más vistas -->
            <div class="wad-card">
                <h2>Top 5 Contenidos Más Vistos</h2>
                <div class="wad-chart-container">
                    <canvas id="mostViewedChart" aria-label="Gráfico de entradas más vistas" role="img"></canvas>
                </div>
            </div>

            <!-- Tarjeta 1: Distribución de Contenido -->
            <div class="wad-card">
                <h2>Distribución de Contenido</h2>
                <div class="wad-chart-container">
                    <canvas id="contentDistributionChart" aria-label="Gráfico de distribución de contenido" role="img"></canvas>
                </div>
            </div>

            <!-- Tarjeta 2: Estado de Comentarios -->
            <div class="wad-card">
                <h2>Estado de los Comentarios</h2>
                <div class="wad-chart-container">
                    <canvas id="commentStatusChart" aria-label="Gráfico de estado de comentarios" role="img"></canvas>
                </div>
            </div>

            <!-- Tarjeta Nueva: Plugins Utilizados -->
            <div class="wad-card">
                <h2>Estado de Plugins</h2>
                <div class="wad-chart-container">
                    <canvas id="pluginsStatusChart" aria-label="Gráfico de estado de plugins" role="img"></canvas>
                </div>
                <!-- Aquí JavaScript insertará la lista en texto -->
                <div class="wad-plugin-list" id="wadPluginList"></div>
            </div>

            <!-- Tarjeta 3: Usuarios por Rol -->
```

```
                <!-- Tarjeta 3: Usuarios por Rol -->
                <div class="wad-card">
                    <h2>Usuarios por Rol</h2>
                    <div class="wad-chart-container">
                        <canvas id="usersByRoleChart" aria-label="Gráfico de usuarios por rol" role="img"></canvas>
                    </div>
                </div>

                <!-- Tarjeta 4: Publicaciones en los últimos 30 días -->
                <div class="wad-card wad-card-wide">
                    <h2>Actividad de Publicaciones (Últimos 30 días)</h2>
                    <div class="wad-chart-container wad-chart-container-large">
                        <canvas id="recentPostsChart" aria-label="Gráfico de publicaciones recientes" role="img"></canvas>
                    </div>
                </div>
            </div>
        </div>
    </div>
    <?php
}
```

5.4. Cargar y ejecutar las instrucciones de los gráficos (JS)

Ahora que tenemos los datos listos y el "artista" (Chart.js) en el escenario, necesitamos darle las instrucciones detalladas de *cómo* debe dibujar cada gráfico. Esto se hace en un archivo independiente para mantener el código limpio y organizado.

¿Qué hacer? Crea una carpeta llamada `assets/js` en la raíz de tu plugin y dentro crea un archivo llamado **dashboard-charts.js**. Pega ahí todo el código JavaScript que desarrollamos anteriormente.

```
1 // Esperar a que el DOM esté completamente cargado
2 document.addEventListener('DOMContentLoaded', function() {
3
4     // Verificamos si los datos han sido pasados desde PHP
5     if (typeof wadChartData === 'undefined') {
6         console.error('No se encontraron los datos para los gráficos.');
```

```
7         return;
8     }
9
10    // Colores globales para mantener un diseño coherente
11    const colors = {
12        primary: '#2271b1', // Azul WordPress
13        secondary: '#00a32a', // Verde
14        accent: '#d63638', // Rojo
15        warning: '#dba617', // Amarillo
16        purple: '#8224e3',
17        cyan: '#00a0d2',
18        gray: '#646970',
19        lightGray: '#f0f0f1'
20    };
21
22    // --- 0. KPI Visitas Totales ---
23    const totalViewsEl = document.getElementById('wadTotalViews');
24    if (totalViewsEl) {
25        // Animación sencilla de contador
26        let current = 0;
27        const target = wadChartData.total_views;
28        const increment = Math.ceil(target / 50); // Ajusta la velocidad
29
30        if (target > 0) {
31            const timer = setInterval(() => {
32                current += increment;
33                if (current ≥ target) {
34                    current = target;
35                    clearInterval(timer);
36                }
37                totalViewsEl.textContent = new Intl.NumberFormat().format(current);
38            }, 20);
39        }
40    }
41}
```



```

81 // --- NUEVO: Gráfico Top 5 Más Vistos (Barra Horizontal) ---
82 const ctxMostViewed = document.getElementById('mostViewedChart');
83 if (ctxMostViewed) {
84   const mvLabels = wadChartData.most_viewed.map(item => {
85     // Cortar títulos muy largos
86     return item.title.length > 25 ? item.title.substring(0, 25) + '...' : item.title;
87   });
88   const mvData = wadChartData.most_viewed.map(item => item.views);
89
90   if (mvData.length === 0) {
91     // Mostrar un mensaje si no hay datos aún
92     ctxMostViewed.style.display = 'none';
93     ctxMostViewed.parentElement.innerHTML += '<p style="text-align:center; color:#646970; margin-top:50px;">Aún no hay visitas registradas. ¡Visita una de tus páginas para ver este gráfico!</p>';
94   } else {
95     new Chart(ctxMostViewed, {
96       type: 'bar',
97       data: {
98         labels: mvLabels,
99         datasets: [{
100           label: 'Visitas',
101           data: mvData,
102           backgroundColor: colors.cyan,
103           borderRadius: 4
104         }]
105       },
106       options: {
107         indexAxis: 'y', // Barra horizontal
108         responsive: true,
109         maintainAspectRatio: false,
110         plugins: {
111           legend: { display: false }
112         },
113         scales: {
114           x: { beginAtZero: true, ticks: { stepSize: 1 } }
115         }
116       }
117     });
118   }
119 }
120

```

```

81
82 // --- NUEVO: Gráfico de Plugins (Doughnut) ---
83 const ctxPlugins = document.getElementById('pluginsStatusChart');
84 const pluginListEl = document.getElementById('wadPluginList');
85 if (ctxPlugins && wadChartData.plugins_data) {
86   new Chart(ctxPlugins, {
87     type: 'doughnut',
88     data: {
89       labels: ['Activos', 'Inactivos'],
90       datasets: [{
91         data: [
92           wadChartData.plugins_data.active_count,
93           wadChartData.plugins_data.inactive_count
94         ],
95         backgroundColor: [colors.primary, colors.lightGray],
96         borderWidth: 1
97       }]
98     },
99     options: {
100       responsive: true,
101       maintainAspectRatio: false,
102       plugins: {
103         legend: { position: 'bottom' }
104       }
105     }
106   });
107
108   // Insertar lista de plugins
109   let listHTML = '';
110   wadChartData.plugins_data.list.forEach(plugin => {
111     const statusClass = plugin.active ? 'wad-plugin-active' : 'wad-plugin-inactive';
112     const statusText = plugin.active ? '✓ Activo' : 'X Inactivo';
113     listHTML += `
114       <div class="wad-plugin-item">
115         <span>${plugin.name}</span>
116         <span class="${statusClass}">${statusText}</span>
117       </div>
118     `;
119   });
120   pluginListEl.innerHTML = listHTML;
121 }
122

```

```

123
124 // --- 1. Gráfico de Distribución de Contenido (Pie) ---
125 const ctxContent = document.getElementById('contentDistributionChart');
126 if (ctxContent) {
127     new Chart(ctxContent, {
128         type: 'pie',
129         data: {
130             labels: ['Entradas (Posts)', 'Páginas (Pages)'],
131             datasets: [{
132                 data: [
133                     wadChartData.content_distribution.posts,
134                     wadChartData.content_distribution.pages
135                 ],
136                 backgroundColor: [colors.primary, colors.secondary],
137                 borderWidth: 1
138             }]
139         },
140         options: {
141             responsive: true,
142             maintainAspectRatio: false,
143             plugins: { legend: { position: 'bottom' } }
144         }
145     });
146 }

```

```

147
148 // --- 2. Gráfico de Estado de Comentarios (Doughnut) ---
149 const ctxComments = document.getElementById('commentStatusChart');
150 if (ctxComments) {
151     new Chart(ctxComments, {
152         type: 'doughnut',
153         data: {
154             labels: ['Aprobados', 'Pendientes', 'Spam'],
155             datasets: [{
156                 data: [
157                     wadChartData.comment_status.approved,
158                     wadChartData.comment_status.pending,
159                     wadChartData.comment_status.spam
160                 ],
161                 backgroundColor: [colors.secondary, colors.warning, colors.accent],
162                 borderWidth: 1
163             }]
164         },
165         options: {
166             responsive: true,
167             maintainAspectRatio: false,
168             plugins: { legend: { position: 'bottom' } }
169         }
170     });
171 }
172

```

```

172
173 // --- 3. Gráfico de Usuarios por Rol (Bar) ---
174 const ctxUsers = document.getElementById('usersByRoleChart');
175 if (ctxUsers) {
176     const roleLabels = Object.keys(wadChartData.users_by_role);
177     const roleData = Object.values(wadChartData.users_by_role);
178
179     new Chart(ctxUsers, {
180         type: 'bar',
181         data: {
182             labels: roleLabels,
183             datasets: [{
184                 label: 'Usuarios',
185                 data: roleData,
186                 backgroundColor: colors.purple,
187                 borderRadius: 4
188             }]
189         },
190         options: {
191             responsive: true,
192             maintainAspectRatio: false,
193             plugins: { legend: { display: false } },
194             scales: { y: { beginAtZero: true, ticks: { stepSize: 1 } } }
195         }
196     });
197 }

```

```

199 // --- 4. Gráfico de Publicaciones Recientes (Line) ---
200 const ctxPosts = document.getElementById('recentPostsChart');
201 if (ctxPosts) {
202     const dates = Object.keys(wadChartData.recent_posts);
203     const postCounts = Object.values(wadChartData.recent_posts);
204
205     const formattedDates = dates.map(dateStr => {
206         const dateObj = new Date(dateStr);
207         return `${dateObj.getDate()}/${dateObj.getMonth() + 1}`;
208     });
209
210     new Chart(ctxPosts, {
211         type: 'line',
212         data: {
213             labels: formattedDates,
214             datasets: [{
215                 label: 'Entradas publicadas',
216                 data: postCounts,
217                 borderColor: colors.primary,
218                 backgroundColor: 'rgba(34, 113, 177, 0.1)',
219                 borderWidth: 2,
220                 fill: true,
221                 tension: 0.3
222             }]
223         },
224         options: {
225             responsive: true,
226             maintainAspectRatio: false,
227             plugins: {
228                 legend: { position: 'top' },
229                 tooltip: {
230                     callbacks: {
231                         title: function(context) { return dates[context[0].dataIndex]; }
232                     }
233                 }
234             },
235             scales: { y: { beginAtZero: true, ticks: { stepSize: 1 } } }
236         }
237     });
238 }
239
240 });

```

Explicación:

document.addEventListener('DOMContentLoaded', ...): Es la regla de oro. Esta línea obliga al navegador a esperar a que toda la estructura (el HTML) esté lista antes de intentar dibujar. Sin esto, el código fallaría porque los lienzos (<canvas>) aún no existirían.

if (typeof wadChartData === 'undefined'): Es un mecanismo de seguridad. Verifica que los datos enviados desde PHP llegaron correctamente. Si por algún motivo el "puente" de datos falló, esta línea evita que el navegador muestre errores técnicos en pantalla.

const colors = { ... }: Definimos nuestra paleta de colores. Centralizar esto aquí nos permite cambiar el diseño de todos los gráficos a la vez simplemente modificando estos códigos HEX, asegurando una estética profesional.

document.getElementById(...): Localiza exactamente en qué parte del HTML (<canvas id="...">) debe "pintar" cada gráfico.

new Chart(ctx, { ... }): Es la instrucción directa a la librería. Aquí configuramos el tipo de gráfico (barras, circular, línea), le pasamos los datos que vienen del servidor (wadChartData) y definimos el comportamiento de los ejes y las leyendas.

wadChartData.xxx.map(...): Es una técnica de transformación. Como los datos vienen de PHP, a veces necesitamos "limpiarlos" o formatearlos (por ejemplo, recortar títulos muy largos o convertir fechas) para que encajen perfectamente en el diseño del gráfico.

pluginListEl.innerHTML = listHTML; Esta línea es la encargada de generar dinámicamente la lista de plugins. En lugar de escribir el HTML a mano, le decimos a JavaScript que construya la lista según los datos reales que encontró, añadiendo etiquetas visuales de "Activo" o "Inactivo".

¿Cómo asegurar que todo esto se ejecute en orden?

Es vital que tu archivo dashboard-charts.js se cargue **después** de Chart.js. En la función que creaste en el punto 5.2, asegúrate de que el tercer parámetro de wp_enqueue_script sea: array('chart-js').

Nota técnica: Al usar array('chart-js'), le estás ordenando a WordPress: *"No cargues mi archivo de instrucciones (dashboard-charts.js) hasta que el 'artista' (Chart.js) esté totalmente cargado y listo"*. Esto evita que aparezca el error *"Chart is not defined"*.

Paso 6: El "Escaparate" (Estilos CSS)

Para que tus gráficos tengan un tamaño adecuado y tu lista de plugins se vea organizada, necesitamos un archivo de estilos.

6.1. ¿Dónde ponerlo?

Crea el archivo en **assets/css/dashboard-style.css**.

6.2. El código CSS

```
1  /*
2  * Estilos para el Dashboard de Actividad
3  */
4
5  .wad-wrap {
6    margin-top: 20px;
7    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Oxygen-Sans, Ubuntu, Cantarell, "Helvetica Neue", sans-serif;
8  }
9
10 .wad-description {
11   font-size: 16px;
12   color: #3c434a;
13   margin-bottom: 20px;
14 }
15
16 .wad-grid {
17   display: grid;
18   grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
19   gap: 20px;
20   margin-top: 20px;
21 }
22
23 .wad-card {
24   background: #fff;
25   border: 1px solid #c3c4c7;
26   border-radius: 4px;
27   box-shadow: 0 1px 1px rgba(0, 0, 0, 0.04);
28   padding: 20px;
29   box-sizing: border-box;
30 }
31
32 .wad-card h2 {
33   margin-top: 0;
34   margin-bottom: 15px;
35   font-size: 18px;
36   color: #1d2327;
37   border-bottom: 1px solid #f0f0f1;
38   padding-bottom: 10px;
39 }
40
41 .wad-chart-container {
42   position: relative;
43   height: 250px;
44   width: 100%;
45 }
```

```

46
47 .wad-chart-container-large {
48     height: 350px;
49 }
50
51 .wad-card-wide {
52     grid-column: 1 / -1;
53 }
54
55 /* Nuevos Estilos para el KPI de Visitas */
56 .wad-kpi-card {
57     display: flex;
58     flex-direction: column;
59     justify-content: center;
60 }
61
62 .wad-kpi-value {
63     font-size: 56px;
64     font-weight: 600;
65     color: #2271b1;
66     text-align: center;
67     margin: 15px 0;
68     line-height: 1;
69 }
70
71 .wad-kpi-desc {
72     text-align: center;
73     color: #646970;
74     font-size: 14px;
75     margin: 0;
76 }
77
78 /* Nuevos Estilos para la Lista de Plugins */
79 .wad-plugin-list {
80     margin-top: 15px;
81     max-height: 150px;
82     overflow-y: auto;
83     border-top: 1px solid #f0f0f1;
84     padding-top: 10px;
85 }
86

```

```

86
87 .wad-plugin-item {
88     font-size: 13px;
89     padding: 6px 0;
90     display: flex;
91     justify-content: space-between;
92     border-bottom: 1px solid #f6f7f7;
93 }
94
95 .wad-plugin-item:last-child {
96     border-bottom: none;
97 }
98
99 .wad-plugin-active {
100     color: #00a32a;
101     font-weight: 600;
102 }
103
104 .wad-plugin-inactive {
105     color: #d63638;
106 }
107
108 @media screen and (max-width: 782px) {
109     .wad-grid {
110         grid-template-columns: 1fr;
111     }
112 }
113

```